Untitled-1.cpp

```cpp
__global__ void transposeOptimized(float *in, float *out, int width, int height) {
    // Shared Memory 做中转;TILE_DIM + 1 也就是 Padding，消除 Bank Conflict!
    __shared__ float tile[TILE_DIM][TILE_DIM + 1];
    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;
    // 技巧 3：合并读取 (Coalesced Read) 从 Global 读入 Shared
    if (x < width && y < height)
        tile[threadIdx.y][threadIdx.x] = in[y * width + x];
    __syncthreads(); // 必须同步，等一个 Block 的数据全部就位
    int trans_x = blockIdx.y * TILE_DIM + threadIdx.x;
    int trans_y = blockIdx.x * TILE_DIM + threadIdx.y;
    if (trans_x < height && trans_y < width)
        out[trans_y * height + trans_x] = tile[threadIdx.x][threadIdx.y];
}
int main() {
    const int numStreams = 4,width = 1024, height = 1024;
    const int elementsPerMatrix = width * height;
    const size_t bytesPerMatrix = elementsPerMatrix * sizeof(float);
    const size_t totalBytes = numStreams * bytesPerMatrix;
    float *h_in, *h_out,*d_in, *d_out;
    // 异步拷贝必须搭配 cudaMallocHost，普通的 malloc 会导致退化为同步等待
    cudaMallocHost((void**)&h_in, totalBytes);
    cudaMallocHost((void**)&h_out, totalBytes);
    cudaMalloc((void**)&d_in, totalBytes);
    cudaMalloc((void**)&d_out, totalBytes);
    for (int i = 0; i < numStreams * elementsPerMatrix; i++)
        h_in[i] = 1.0f;
    cudaStream_t streams[numStreams];
    for (int i = 0; i < numStreams; i++)
        cudaStreamCreate(&streams[i]);
    dim3 block(TILE_DIM, TILE_DIM);
    dim3 grid((width + TILE_DIM - 1) / TILE_DIM, (height + TILE_DIM - 1) / TILE_DIM);
    for (int i = 0; i < numStreams; i++) {
        int offset = i * elementsPerMatrix;
        // 1. 异步拷贝：Host -> Device (传入 streams[i])
        cudaMemcpyAsync(&d_in[offset], &h_in[offset], bytesPerMatrix, cudaMemcpyHostToDevice, streams[i]);
        // 2. 执行计算 Kernel (配置的第4个参数传入 streams[i]，第3个参数0代表不使用动态共享内存)
        transposeOptimized<<<grid, block, 0, streams[i]>>>(&d_in[offset], &d_out[offset], width, height);
        // 3. 异步拷贝：Device -> Host (传入 streams[i])
        cudaMemcpyAsync(&h_out[offset], &d_out[offset], bytesPerMatrix, cudaMemcpyDeviceToHost, streams[i]);
    }
    cudaDeviceSynchronize();
    for (int i = 0; i < numStreams; i++)
        cudaStreamDestroy(streams[i]);// 销毁流释放内存
    cudaFreeHost(h_in);cudaFreeHost(h_out);
    cudaFree(d_in);cudaFree(d_out);
}
```